# Raspberry Pi verstehen: Schritt für Schritt

Stand: Oktober 2026 · Version 67

Dieses Lernheft zeigt dir, was ein Raspberry Pi ist, wie du ihn in Betrieb nimmst und wie du mit Python LEDs, Taster und Sensoren steuerst. Alle Beispiele kannst du direkt im **Pi-Lab** ausprobieren (Knöpfe „Probier es aus“ und „Aufgabe“). Am echten Pi funktionieren dieselben Programme. Das letzte Kapitel vergleicht Arduino und Pi.

**Inhalt**


1. Was ist ein Raspberry Pi?
2. Inbetriebnahme
3. Die GPIO-Leiste
4. Python in fünf Minuten
5. Erstes Projekt: LED und Taster
6. Mehr Bauteile
7. Projektideen für den Pi
8. Fehler finden und sicher arbeiten
9. Arduino oder Pi? Und wie geht es hier weiter


## 1. Was ist ein Raspberry Pi?

Ein **Raspberry Pi** ist ein vollständiger kleiner **Computer** in der Größe einer Kreditkarte. Er hat Prozessor, Arbeitsspeicher, USB, HDMI, WLAN und läuft mit einem **Betriebssystem** (Raspberry Pi OS, ein Linux). Das Programm liegt nicht fest im Chip, sondern auf einer **SD-Karte**, und du startest es wie auf einem PC.

| | Arduino Uno | Raspberry Pi |
|---|---|---|
| Art | Mikrocontroller | Einplatinen-Computer |
| Betriebssystem | keins, ein Programm | Linux, viele Programme gleichzeitig |
| Sprache | C++ (Sketch) | meist Python |
| Pin-Spannung | **5 V** | **3,3 V** |
| Analoge Eingänge | ja (A0 bis A5) | **nein** (ADC-Baustein nötig) |
| Zeitgenauigkeit | sehr genau, in Echtzeit | nicht garantiert (Linux läuft nebenher) |
| Strombedarf | ~50 mA | 500 mA bis 1,5 A, eigenes Netzteil |
| Starten | sofort | braucht Sekunden bis Minuten |
| Stärke | Sensoren und Motoren direkt steuern | Netzwerk, Kamera, Bildschirm, Datenbanken, KI |

> **Merke:** Arduino = Steuern in Echtzeit. Pi = Rechnen, Vernetzen, Anzeigen. Beide arbeiten oft zusammen: Der Arduino liest den Sensor, der Pi wertet aus und zeigt an.

Zur Abgrenzung: Der **Raspberry Pi Pico** ist kein Pi-Computer, sondern ein Mikrocontroller wie der Arduino (Programmierung mit MicroPython oder C++).

## 2. Inbetriebnahme

### Was du brauchst

- Raspberry Pi (zum Beispiel Pi 4 oder Pi 5) mit passendem **Netzteil** (Pi 4: 5 V / 3 A USB-C, Pi 5: 5 V / 5 A)
- **microSD-Karte** (mindestens 16 GB)
- Entweder Monitor (Micro-HDMI), Tastatur und Maus, oder nur WLAN und einen zweiten Computer (*headless*)

### Betriebssystem auf die SD-Karte

Eine ausführliche Anleitung mit Programm-Download steht auf der Seite SD-Karte vorbereiten.

1. Auf einem Computer das kostenlose Programm **Raspberry Pi Imager** installieren (raspberrypi.com/software).
2. Gerät (Raspberry Pi 4/5 …) und **Raspberry Pi OS** (64-Bit) wählen, SD-Karte auswählen.
3. Vor dem Schreiben auf **Einstellungen bearbeiten** klicken: Hostname (zum Beispiel `pi-lab1`), Benutzername und Passwort, WLAN-Zugang, Zeitzone und **SSH aktivieren**.
4. Schreiben lassen, SD-Karte in den Pi stecken, Netzteil anschließen. Der erste Start dauert 1 bis 2 Minuten.

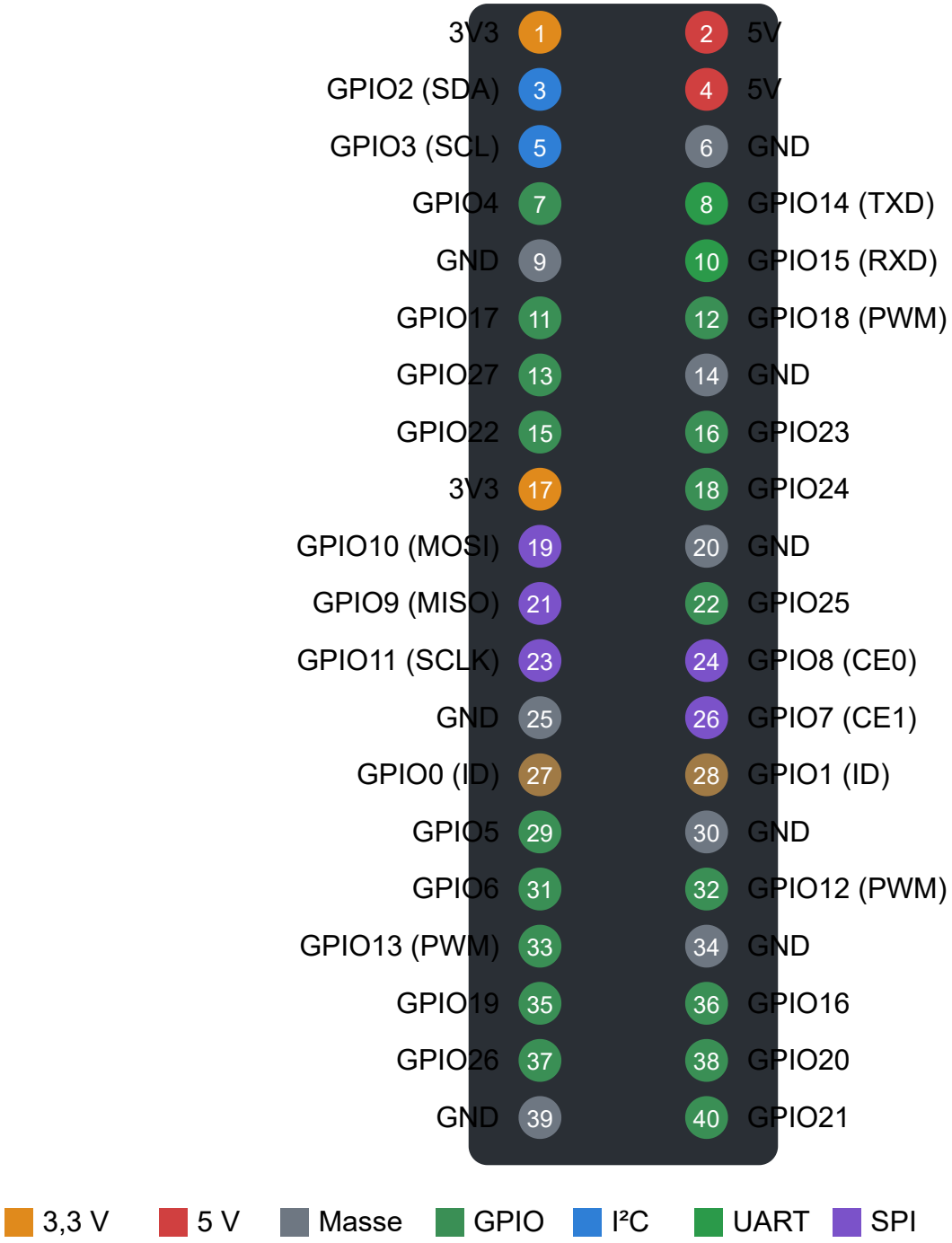
### Verbinden

- **Mit Monitor:** Der Desktop startet. Du findest dort das Terminal und die Programmierumgebung **Thonny** (Menü Entwicklung).
- **Ohne Monitor (SSH):** Vom Computer oder **iPad** mit einer SSH-App (zum Beispiel Termius oder Blink) verbinden: `ssh benutzername@pi-lab1.local`.
- **Fernsteuerung des Desktops:** Mit „Raspberry Pi Connect" oder einer VNC-App kannst du den Desktop auch am iPad anzeigen.

> **Wichtig:** Den Pi nie einfach vom Strom trennen, das kann die SD-Karte beschädigen. Erst herunterfahren: `sudo shutdown now` (oder im Desktop-Menü „Herunterfahren"), dann nach einigen Sekunden Netzteil abziehen.

## 3. Die GPIO-Leiste

Über die 40-polige Stiftleiste (**GPIO** = General Purpose Input/Output) verbindet der Pi sich mit der Elektronik. Die meisten Pins sind frei belegbare Ein-/Ausgänge, einige haben Sonderfunktionen ($I^2C$, UART, SPI, PWM).




### Zwei Zählweisen

Jeder Pin hat eine **Pin-Nummer** (1 bis 40, wie auf der Leiste) und eine **GPIO-Nummer** (BCM). Das Python-Modul `gpiozero` arbeitet mit den **GPIO-Nummern**. `LED(17)` meint GPIO17, das ist Pin 11 der Leiste.

> **Die Pins vertragen nur 3,3 V!** Ein 5-V-Signal (zum Beispiel vom Echo-Pin eines Ultraschallsensors oder von einem Arduino) kann den Pi zerstören. Dafür braucht man einen Spannungsteiler oder einen Pegelwandler. Pro Pin sind etwa **16 mA** erlaubt, bleib bei LEDs mit 330 Ω Vorwiderstand (3,3 V − 2 V = 1,3 V, 4 mA bis 10 mA reichen für eine helle LED).

Die Pins 1 und 17 liefern 3,3 V, die Pins 2 und 4 liefern 5 V (vom Netzteil, nur als Stromversorgung, nicht als Signal). Masse (GND) gibt es an mehreren Pins, zum Beispiel Pin 6 und Pin 9.

## 4. Python in fünf Minuten

Auf dem Pi programmierst du meist in **Python**. Das liest sich fast wie Englisch. Statt geschweifter Klammern zählt die **Einrückung** (4 Leerzeichen).

```
name = "Azubi"                    # Variable (Text)
zahl = 7                          # ganze Zahl
print("Hallo", name, zahl)        # Ausgabe

if zahl > 5:                      # Entscheidung (Doppelpunkt!)
    print("groß")
else:
    print("klein")

for i in range(3):                # i = 0, 1, 2
    print(i)

while True:                       # Endlosschleife
    break                         # verlässt die Schleife

def verdopple(x):                 # eigene Funktion
    return x * 2
```

- **Doppelpunkt** am Ende von `if`, `for`, `while`, `def`.
- **Einrückung** zeigt, was zum Block gehört. Falsch eingerückt gibt einen Fehler.
- Warten: `from time import sleep` und `sleep(0.5)` (Sekunden, auch Kommazahlen).
- Ein Programm startest du in Thonny mit dem grünen Pfeil oder im Terminal mit `python3 programm.py`. Beenden mit `Strg+C`.

✎ Aufgabe: Hallo im Terminal

### Eigene Klassen: ein Bauplan für Objekte

Brauchst du mehrere gleiche Dinge, zum Beispiel drei LEDs, die jeweils anders blinken, schreibst du einen **Bauplan**: eine **Klasse**. Aus dem Bauplan baust du beliebig viele **Objekte**.

```
class Blinker:                        # der Bauplan
    def __init__(self, pin):          # wird beim Erzeugen aufgerufen
        self.led = LED(pin)           # self = dieses Objekt

    def start(self, takt):            # eine Methode
        self.led.blink(on_time=takt, off_time=takt)

rot = Blinker(18)                     # Objekt 1
```

```
gelb = Blinker(24)                  # Objekt 2
rot.start(0.5)
gelb.start(0.2)
```

- Jede Methode hat als ersten Parameter `self`. Damit greift sie auf die Daten des eigenen Objekts zu (`self.led`).
- `__init__` ist die Startmethode: Hier legst du an, was jedes Objekt braucht.
- Eine Klasse kann von einer anderen **erben** (`class Schnell(Blinker):`) und mit `super()` deren Methoden nutzen. Im Pi-Lab geht das nur mit eigenen Klassen, nicht mit `LED` selbst.

▶ Eigene Klasse: zwei Blinker | ✎ Aufgabe: Eigene Klasse: der Blinker

## 5. Erstes Projekt: LED und Taster

Die Bibliothek **gpiozero** ist auf dem Raspberry Pi OS schon installiert und macht das Ansteuern sehr einfach.

### LED blinken lassen

Schaltung: GPIO17 (Pin 11) → 330-Ω-Widerstand → LED (Anode) → Kathode an GND (zum Beispiel Pin 9).

```
from gpiozero import LED
from time import sleep

led = LED(17)
while True:
    led.on()
    sleep(1)
    led.off()
    sleep(1)
```

Kurz: `led.blink()` blinkt im Hintergrund, danach mit `pause()` (aus `signal`) warten.

### Taster abfragen

Schaltung: Taster zwischen GPIO2 (Pin 3) und GND. `Button` schaltet intern einen Pull-up ein, das ist wie `INPUT_PULLUP` beim Arduino.

```
from gpiozero import Button
from time import sleep

taster = Button(2)
while True:
    if taster.is_pressed:        # hier: True = gedrückt (gpiozero dreht das für dich um)
        print("gedrückt")
    sleep(0.1)
```

### Taster schaltet LED (ereignisgesteuert)

```
from gpiozero import LED, Button
from signal import pause

led = LED(17)
taster = Button(2)
taster.when_pressed = led.on
taster.when_released = led.off
pause()                          # Programm läuft weiter und wartet auf Ereignisse
```

▶ LED blinkt ▶ Taster schaltet ▶ Taster abfragen ▶ LED mit RPi.GPIO (klassisch)

✎ Aufgabe: Dreimal blinken ✎ Aufgabe: Taster schaltet die LED ✎ Aufgabe: Tastendrücke zählen

> **Unterschied zum Arduino:** `is_pressed` ist True, wenn der Taster gedrückt ist. `gpiozero` kümmert sich um den Pull-up. Beim Arduino musst du selbst LOW abfragen.

## 6. Mehr Bauteile

### LED dimmen (PWM)

```
from gpiozero import PWMLED
from time import sleep
led = PWMLED(18)
led.value = 0.5          # 50 % Helligkeit (0.0 bis 1.0)
led.pulse()              # sanftes Pulsieren im Hintergrund
sleep(10)
```

### Analoge Sensoren: der Pi hat keinen ADC

Ein Poti, ein Fotowiderstand oder ein TMP36 liefern eine **Spannung**. Der Arduino misst sie mit `analogRead`. Der Pi hat dafür **keinen** Eingang. Du brauchst einen kleinen Wandler-Baustein, zum Beispiel den **MCP3008** (8 Kanäle, 10 Bit). Er misst die Spannung an einem Kanal und schickt sie als Zahl über vier Leitungen (**SPI**) zum Pi.

| MCP3008-Anschluss | Wohin |
|---|---|
| VDD und VREF | 3,3 V (nicht 5 V!) |
| AGND und DGND | Masse |
| CLK (Takt) | GPIO11 (SCLK) oder ein frei gewählter Pin |
| DOUT (Daten zum Pi) | GPIO9 (MISO) oder frei |
| DIN (Daten vom Pi) | GPIO10 (MOSI) oder frei |
| CS (Chip-Select) | GPIO8 (CE0) oder frei |
| CH0 bis CH7 | die Sensoren (Spannung zwischen 0 und 3,3 V) |

```
from gpiozero import MCP3008

# Hardware-SPI (feste Pins GPIO11, 9, 10, 8): SPI muss am Pi eingeschaltet sein
poti = MCP3008(channel=0)

# oder Software-SPI mit frei gewählten Pins
poti = MCP3008(channel=0, clock_pin=23, miso_pin=24, mosi_pin=25, select_pin=8)

print(poti.value)          # 0.0 bis 1.0
print(poti.raw_value)      # 0 bis 1023
print(poti.voltage)        # in Volt (0 bis 3,3)
```

> **Achtung:** Am Eingang dürfen höchstens 3,3 V liegen. Der TMP36 liefert bei 3,3 V Versorgung bis etwa 1,5 V und passt gut. Ein Sensor mit 5 V Ausgang braucht einen Spannungsteiler.

Beim TMP36 gilt wie beim Arduino: `celsius = (volt - 0.5) * 100`.

▶ Poti lesen ▶ Helligkeit messen ▶ Temperatur messen ✎ Aufgabe: Poti über den MCP3008 lesen ✎ Aufgabe: Dunkelheits-Anzeige ✎ Aufgabe: Temperatur ausgeben ✎ Aufgabe: Der Wandler antwortet nicht

### Ultraschall-Abstand

> Der Echo-Pin des HC-SR04 liefert 5 V. Vor den Pi gehört ein **Spannungsteiler** (zum Beispiel 1 kΩ und 2 kΩ), sonst wird der Pin zerstört.

```
from gpiozero import DistanceSensor
from time import sleep
sensor = DistanceSensor(echo=24, trigger=23, max_distance=2)
while True:
    print(round(sensor.distance * 100), "cm")
    sleep(0.5)
```

### Servo

```
from gpiozero import Servo
from time import sleep
servo = Servo(17)
servo.min(); sleep(1)
servo.mid(); sleep(1)
servo.max(); sleep(1)
```

▶ LED pulsiert ▶ Ultraschall ▶ Servo ✎ Aufgabe: LED pulsiert ✎ Aufgabe: Abstandswarner ✎ Aufgabe: Servo auf 90 Grad

### Sensoren mit I²C

Viele Sensoren (Temperatur, Licht, Display, Beschleunigung) sprechen **I²C**. Dafür reichen zwei Signalleitungen für beliebig viele Geräte: **SDA** (Daten, GPIO2, Pin 3) und **SCL** (Takt, GPIO3, Pin 5), dazu 3,3 V und Masse. Jedes Gerät hat eine feste **Adresse**, so weiß der Pi, wen er anspricht. Die nötigen Pull-up-Widerstände sind beim Pi schon eingebaut.

> **Merke:** Auf dem echten Pi muss I²C erst eingeschaltet werden: „Raspberry Pi Konfiguration → Schnittstellen → I2C" oder `sudo raspi-config`. Mit `i2cdetect -y 1` findest du angeschlossene Geräte und ihre Adressen.

```
from smbus2 import SMBus

bus = SMBus(1)                                  # Bus 1 = GPIO2 und GPIO3
daten = bus.read_i2c_block_data(0x48, 0, 2)    # Adresse 0x48, Register 0, 2 Bytes lesen
print(daten)                                    # zum Beispiel [25, 0]
```

Die Zahlen mit `0x` sind **Hexadezimalzahlen** (0x48 ist 72). Jeder Sensor liefert seine Messwerte in Bytes. Wie du sie umrechnest, steht im Datenblatt. Beim Temperatursensor LM75 sind es 9 Bit in zwei Bytes, ein Schritt sind 0,5 Grad. Beim Lichtsensor BH1750 sind es 16 Bit, geteilt durch 1,2 ergibt Lux.

| Fehlermeldung | Bedeutung |
|---|---|
| `[Errno 121] Remote I/O error` | Kein Gerät antwortet: Versorgung, SDA, SCL oder die Adresse stimmen nicht. Im Pi-Lab sagt der Tipp in der Meldung, was fehlt. |
| `[Errno 2] No such file or directory: '/dev/i2c-1'` | I²C ist auf dem Pi nicht eingeschaltet (oder falsche Busnummer). |

▶ Temperatur über I ▶ Lichtsensor BH1750 ▶ Zwei Sensoren am selben ✎ Aufgabe: Temperatur über I²C lesen ✎ Aufgabe: Dunkelheits-LED über I²C ✎ Aufgabe: Der I²C-Sensor antwortet nicht

## 7. Projektideen für den Pi

- **Ampel** mit drei LEDs und Fußgängertaster (`LEDBoard`, `Button`).
- **Einbruchalarm**: PIR-Sensor löst Summer aus und schreibt die Uhrzeit in eine Datei.
- **Temperaturlogger**: Sensor misst jede Minute und speichert in eine CSV-Datei, die du in Excel öffnest.
- **Webserver**: Eine kleine Webseite zeigt den Sensorwert (Bibliothek `flask`).

- **Kamera**: Fotos per Knopfdruck oder Zeitraffer.
- **Zusammen mit dem Arduino**: Der Arduino misst und sendet per USB-Kabel (Serial), der Pi zeigt die Werte an oder speichert sie.

▶ Ampel ▶ Summer ▶ Bewegungsmelder ▶ Lauflicht ✎ Aufgabe: Ampel

✎ Aufgabe: Summer piept

### Melodien mit dem TonalBuzzer

Ein **aktiver** Summer kann nur einen festen Ton. Mit einem **passiven** Summer oder Lautsprecher kannst du die Tonhöhe selbst bestimmen: `TonalBuzzer` spielt Noten (`"A4"`, `"C#5"`) oder Frequenzen in Hz. Mit `octaves=2` reicht der Bereich über zwei Oktaven nach oben und unten (von A2 bis A6).

▶ Star-Wars-Thema mit Lichtshow

### Programm beim Start automatisch ausführen

Im Terminal `crontab -e` öffnen und am Ende eine Zeile einfügen: `@reboot python3 /home/benutzername/projekt.py &`. Beim nächsten Neustart läuft das Programm von selbst.

## 8. Fehler finden und sicher arbeiten

| Symptom | Meist die Ursache |
|---|---|
| LED leuchtet nicht | falsche GPIO-Nummer (BCM statt Pin-Nummer!), LED falsch herum, Masse nicht verbunden |
| `GPIO busy` / „Pin already in use“ | Ein anderes Programm (oder ein zweites Fenster) nutzt den Pin noch. Beenden mit Strg+C. |
| `ModuleNotFoundError` | Bibliothek fehlt: `sudo apt install python3-gpiozero` |
| Einrückungsfehler | Tabulator und Leerzeichen gemischt; Doppelpunkt vergessen |
| Pi startet nicht | Netzteil zu schwach, SD-Karte falsch beschrieben. Auf der SD-Karte mit dem Imager neu beginnen. |
| Sensor liefert Unsinn | 5-V-Sensor an 3,3-V-Pin, fehlender Spannungsteiler oder fehlende Masse |

✎ Aufgabe: Die LED bleibt dunkel ✎ Aufgabe: Fehlermeldung lesen

✎ Aufgabe: Ultraschall zerstört den Pi

> **Regeln für sicheres Arbeiten:**
>
> - Verdrahten nur bei **heruntergefahrenem Pi**, danach Verbindungen doppelt prüfen.

- Kein 5-V-Signal auf einen GPIO-Pin. Nie 3,3 V und 5 V verbinden.
- Pins nie gegeneinander oder gegen Masse kurzschließen.
- LEDs immer mit Vorwiderstand, Motoren und Lampen nur über Transistor, Relais oder Treiberbaustein.

## 9. Arduino oder Pi? Und wie geht es hier weiter

| Aufgabe | Besser geeignet |
|---|---|
| Sensor auslesen, LED/Motor direkt schalten | Arduino |
| Zeitkritisch, schnell reagieren | Arduino |
| Strom sparen, Batteriebetrieb | Arduino |
| Daten speichern, Webseite, Netzwerk, Kamera | Raspberry Pi |
| Viele Programme, Bildschirm, Maus | Raspberry Pi |
| Beides kombinieren | Arduino misst, Pi wertet aus (USB/Serial) |

**Im Tonfunk Lab:** Das **Pi-Lab** simuliert einen Raspberry Pi mit Python (gpiozero und RPi.GPIO). Du verdrahtest den Pi an der 40-poligen Leiste, schreibst dein Programm und siehst LEDs, Taster und Sensoren reagieren. Die Pins sind 3,3-V-Pins: Wer 5 V an einen GPIO legt, zerstört den Pi (mit Erklärung).

▶ Pi-Lab öffnen

**Grenzen:** Unterstützt sind print, Schleifen, Funktionen, eigene Klassen (mit Vererbung), with, Listen, Texte, f-Strings, try/except sowie die Module time, random, math, signal, smbus2 ($I^2C$), gpiozero (LED, PWMLED, Buzzer, TonalBuzzer, Button, MotionSensor, DistanceSensor, Servo, AngularServo, LEDBoard, TrafficLights, MCP3008) und RPi.GPIO. Nicht unterstützt sind input(), Dateien, __str__ in eigenen Klassen und Zusatzmodule wie numpy oder flask. Analoge Sensoren über den Wandler MCP3008 (SPI) sind dabei. Nicht unterstützt sind $I^2C$-Geräte.

Weiter geht es mit dem Lernheft Arduino verstehen.